Árvores binárias de busca

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Algoritmos e Estruturas de Dados II
Prof. Fabrício Sérgio de Paula

Tópicos

- Introdução
- Conceitos básicos, busca e inserção
- Mínimo, máximo, sucessor e predecessor
- Remoção
- Exercícios

Introdução

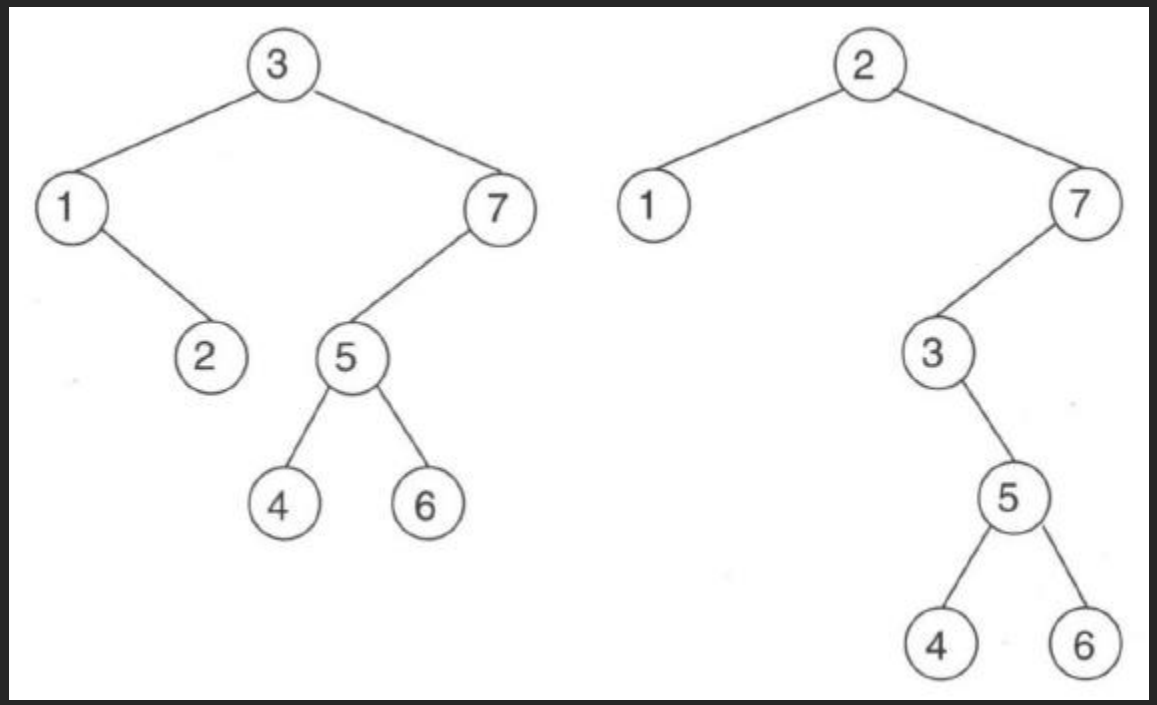
- Busca:
  - Lista:
    - Sem ordenação: O(n)
    - Ordenada: O(lg n) com alocação sequencial ou O(n) com alocação encadeada
  - Árvore binária:
    - Necessário percurso: O(n)
- Árvore binária de busca: estrutura de dados mais adequada para o problema de busca

Conceitos básicos, busca e inserção

- Uma árvore binária de busca T para o conjunto de chaves distintas $S = \{s_1, ..., s_n\}$ possui as seguintes características:
  - T é uma árvore binária com n nós, onde cada nó corresponde a uma chave distinta de S
  - Sejam v, v_1 e v_2 nós de T:
    - Se v1 pertence à subárvore esquerda de v, então $chave(v_1) < chave(v)$
    - Se v_2 pertence à subárvore direita de v, então $chave(v_2) > chave(v)$

Conceitos básicos, busca e inserção

- Exemplos de árvores binárias de busca para S = {1, 2, 3, 4, 5, 6, 7}:



Conceitos básicos, busca e inserção

- Busca por x em uma árvore binária de busca:
  - Iniciar caminho pela raiz
  - Se x = chave do nó atual, busca termina
  - Senão, se x < chave do nó atual, busca continua pela subárvore esquerda
  - Senão (x > chave do nó atual), busca continua pela subárvore direita
  - Busca continua enquanto elemento não foi encontrado e subárvore é não vazia

Conceitos básicos, busca e inserção

- Algoritmo de busca: retorna pt (ponteiro para nó) e f
  - f = 0 (árvore vazia), f = 1 (pt aponta para nó), f = 2 (não foi encontrado à esq.) ou f = 3 (não foi encontrado à dir.)

```
algoritmo 4.1: Busca em árvore binária de busca
procedimento busca-arvore(x, pt, f)
    se pt = λ então f := 0
    senão se x = pt ↑ .chave então  f := 1
          senão se x < pt ↑ .chave então
                    se pt ↑ .esq = λ então f := 2
                    senão pt := pt ↑ .esq
                          busca-arvore(x, pt, f)
                senão se pt ↑ .dir = λ então f := 3
                    senão pt := pt ↑ .dir
                          busca-arvore(x, pt, f)

pt := ptraiz;   busca-arvore(x, pt, f)
```

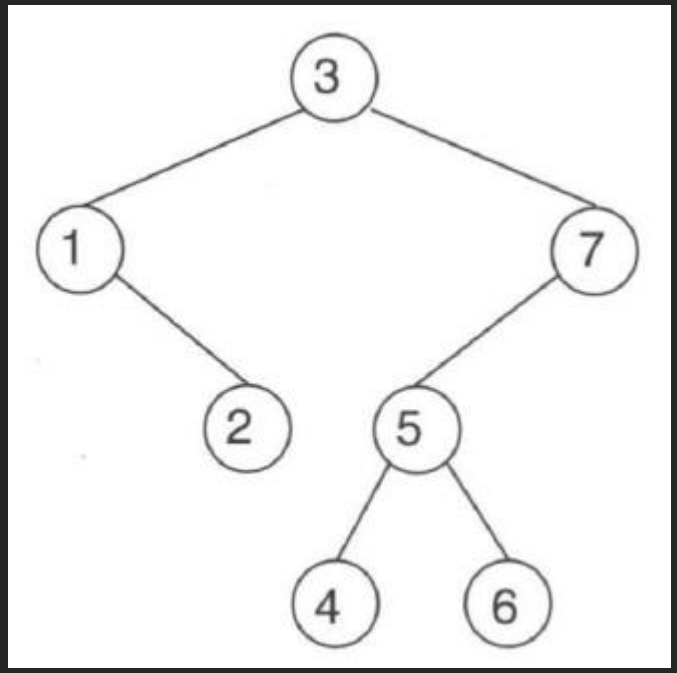
Conceitos básicos, busca e inserção

- Algoritmo de inserção:

```
algoritmo 4.2: Inserção em árvore binária de busca.
pt := ptraiz;   busca-arvore(x, pt, f)
se f = 1 então
        "inserção inválida"
senão ocupar(pt1)
        pt1 ↑ .chave := x;   pt1 ↑ .info := novo-valor
        pt1 ↑ .esq := λ;   pt1 ↑ .dir := λ
        se f = 0 então ptraiz := pt1
        senão se f = 2 então
                        pt ↑ .esq := pt1
                senão pt ↑ .dir := pt1
```

Conceitos básicos, busca e inserção

- Árvore construída através da inserção dos elementos 3, 1, 7, 2, 5, 4, 6:



Conceitos básicos, busca e inserção

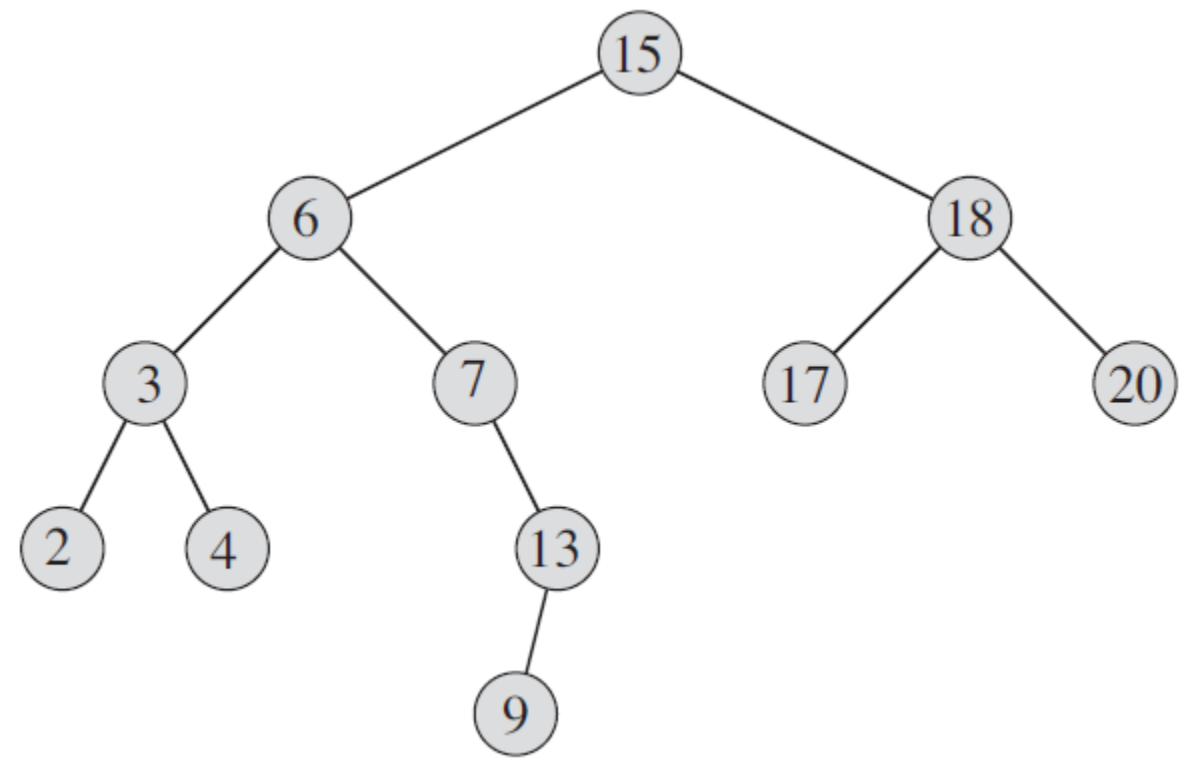
- Complexidade da busca e inserção: O(h)
  - Altura de uma árvore binária de busca:
    - Ziguezague: O(n)
    - Completa: O(lg n)
- Complexidade da construção de uma árvore: n inserções
  - Custo total entre $O(n^2)$ e O(n lg n)

Mínimo, máximo, sucessor e predecessor

- Mínimo: último elemento caminhando pela esquerda
- Máximo: último el emento caminhando pela direita
- Sucessor de x: dois casos
  - Subárvore direita de x não vazia: mínimo da subárvore
  - Subárvore direita de x vazia: ancestral mais baixo cujo filho da esquerda é também ancestral de x
- Predecessor: solução simétrica ...

Mínimo, máximo, sucessor e predecessor

- Mínimo? Máximo? Sucessor de 15? Sucessor de 13?



Remoção

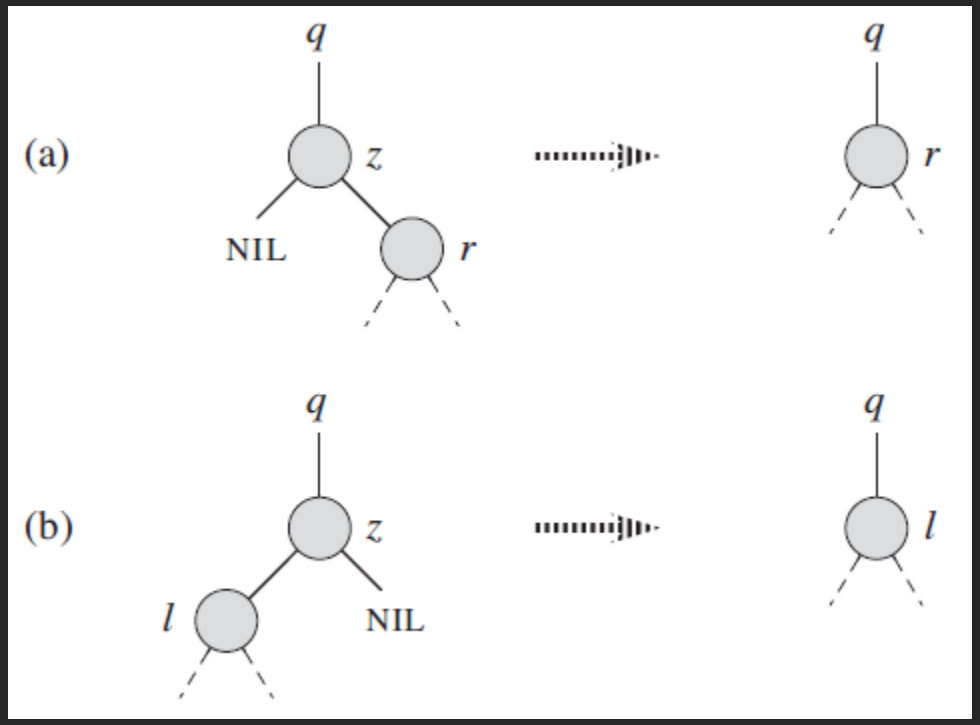
- Idéia geral da remoção do nó z:
  - Se z não tem filhos, simplesmente remova-o
  - Se z tem apenas um filho, utilize-o para substituir z
  - Se z tem dois filhos, encontre y (sucessor de z, que está na subárvore direita de z) e utilize-o para substituir z
    - A subárvore direita restante de z torna-se subárvore direita de y
      - Dois casos a analisar: se y é filho esquerdo ou não
    - A subárvore esquerda de z torna-se subárvore esquerda de y

Remoção

- Situações consideradas pelo algoritmo de remoção de z:
  a) Se z não tem filho esquerdo: z é substituído pelo seu filho direito (pode ser λ ou não)
  b) Se z tem apenas o filho esquerdo: z é substituído pelo filho esquerdo
  c) e d) Se z tem ambos os filhos:
    1. Encontre y, sucessor de z, que está na subárvore direita de z e não tem filho esquerdo
    2. Se y é o filho direito de z, substitua z por y mantendo a subárvore direita de y: (c)
    3. Se y não é filho direito de z, y é substituído pela sua subárvore direita e z é substituído por y: (d)

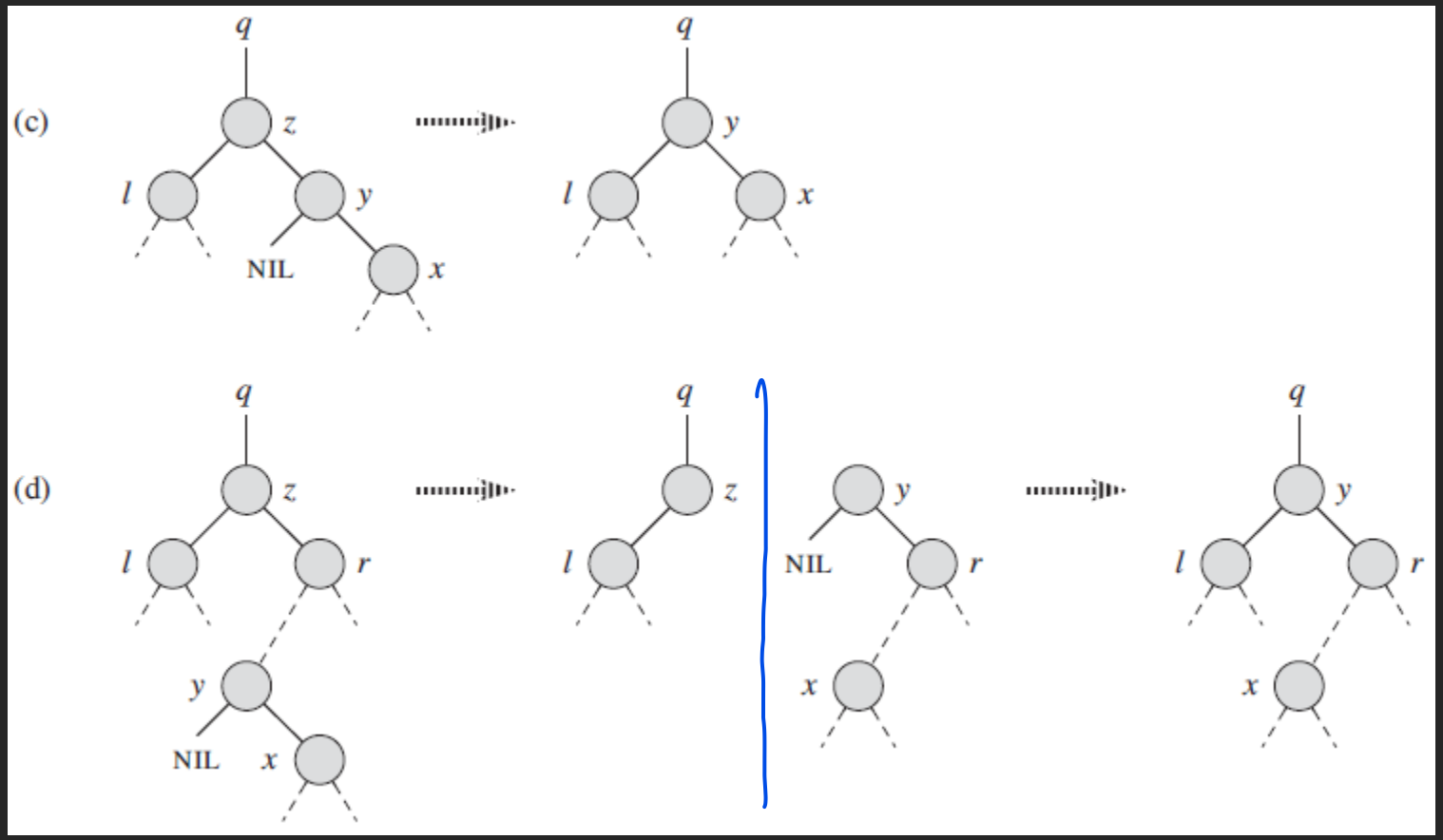
Remoção

- Casos a) e b) da remoção do nó z:



Remoção

- Casos c) e d) da remoção do nó z:



Remoção

- Algoritmo de remoção Tree-Delete:

```
TREE-DELETE(T, z)
if z.left == NIL
    TRANSPLANT(T, z, z.right)
elseif z.right == NIL
    TRANSPLANT(T, z, z.left)
else y = TREE-MINIMUM(z.right)
    if y.p ≠ z
        TRANSPLANT(T, y, y.right)
        y.right = z.right
        y.right.p = y
    TRANSPLANT(T, z, y)
    y.left = z.left
    y.left.p = y
```

Remoção

- Procedimento auxiliar Transplant:
  - Transplant(T, u, v): substitui a subárvore com raiz em u pela subárvore com raiz em v

```
TRANSPLANT(T, u, v)
if u.p == NIL
    T.root = v
elseif u == u.p.left
    u.p.left = v
else u.p.right = v
if v ≠ NIL
    v.p = u.p
```

Exercícios

1. Apresente uma sequência de inserções que produz uma árvore binária de busca completa para o conjunto S = {1, 2, 3, 4, 5, 6, 7}.
2. Apresente uma sequência de inserções que produz uma árvore binária de busca ziguezaque para o mesmo conjunto S.
3. Faça um algoritmo que liste os elementos de uma árvore binária de busca ordenados.
4. Faça algoritmos para encontrar o mínimo e o máximo em uma árvore binária de busca.
5. Apresente o pseudocódigo de um algoritmo para encontrar o sucessor de um nó em uma árvore binária de busca.
6. Apresente o pseudocódigo de um algoritmo para encontrar o predecessor de um nó em uma árvore binária de busca.
7. Descreva a comlexidade de pior caso para cada algoritmo desenvolvido.